

Análisis Comparativo de Modelos de Deep Learning: DNN vs CNN en CIFAR-10

Universitario: Choque Cachi Vismark Abner

Materia: Aprendizaje Profundo (Deep Learning)

Sigla: DAT-264

Docente: Ph.D. Moises Martin Silva Choque

Resumen—El objetivo de este informe es presentar un análisis comparativo entre dos arquitecturas fundamentales de aprendizaje profundo: Redes Neuronales Profundas (DNN) y Redes Neuronales Convolucionales (CNN). El estudio se ha realizado utilizando el conjunto de datos CIFAR-10. Se han implementado y entrenado ambos modelos utilizando los frameworks de PyTorch y TensorFlow/Keras, con el fin de evaluar no solo el rendimiento de las arquitecturas, sino también la eficiencia de las herramientas utilizadas. Se discuten los resultados, ventajas y limitaciones de cada enfoque, proporcionando recomendaciones basadas en la evidencia experimental.

Index Terms—Deep Learning, DNN, CNN, CIFAR-10, PyTorch, TensorFlow, Comparación de Frameworks

I. INTRODUCCIÓN

El objetivo de este informe es presentar un análisis comparativo entre dos arquitecturas fundamentales de aprendizaje profundo: Redes Neuronales Profundas (DNN) y Redes Neuronales Convolucionales (CNN). Estas arquitecturas se utilizan ampliamente para tareas de clasificación de imágenes.

El estudio se ha realizado utilizando el conjunto de datos CIFAR-10, que consiste en 60,000 imágenes a color de 32×32 píxeles, distribuidas en 10 clases diferentes. Se han implementado y entrenado ambos modelos utilizando PyTorch y TensorFlow/Keras, buscando no solo medir el rendimiento de cada arquitectura, sino también observar cómo los diferentes frameworks afectan la eficiencia y la facilidad de entrenamiento.

Este informe ofrece una descripción detallada de las arquitecturas, los métodos de entrenamiento, los resultados obtenidos y un análisis comparativo basado en la precisión y el tiempo de entrenamiento.

II. METODOLOGÍA Y ARQUITECTURAS

Para garantizar una comparación justa, se aplicaron técnicas modernas de regularización y optimización de manera similar en ambos modelos, intentando que las condiciones de entrenamiento fueran lo más equivalentes posible para PyTorch y TensorFlow.

II-A. Deep Neural Network (DNN)

La DNN implementada es un Perceptron Multicapa (MLP) profundo con las siguientes características:

- **Entrada:** Imágenes aplanadas (Flatten) de 3072 dimensiones ($32 \times 32 \times 3$).

- **Estructura:** Bloques densos con conexiones residuales. Las capas ocultas reducen progresivamente su tamaño: $1024 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64$.

- **Justificación de Diseño:**

- **Conexiones Residuales:** Se incluyen saltos entre capas no adyacentes para mitigar el problema del desvanecimiento del gradiente, permitiendo entrenar redes más profundas de manera estable.
- **Batch Normalization:** Se aplica después de cada capa densa para normalizar las activaciones, lo que ayuda a acelerar la convergencia y estabilizar el aprendizaje.
- **Inicialización Kaiming He:** Los pesos iniciales se asignan usando una distribución escalada según el número de neuronas de entrada, lo que mantiene estables las activaciones y evita saturación de gradientes al inicio.

II-B. Convolutional Neural Network (CNN)

La CNN fue diseñada para procesar imágenes de manera jerárquica, detectando primero detalles simples como bordes y colores, y luego combinándolos para reconocer formas y objetos más complejos.

- **Estructura:** Tres bloques convolucionales principales, con profundidad de canales creciente: $32 \rightarrow 64 \rightarrow 128$.
- **Pooling:** MaxPooling (2x2) para reducir dimensionalidad y mantener las características más importantes.

- **Justificación de Diseño:**

- **Extracción de Características:** Los bloques residuales convolucionales permiten aprender representaciones complejas sin perder información espacial prematuramente.
- **MaxPooling vs AvgPooling:** MaxPooling mostró mejor generalización, mientras que AvgPooling tendía a sobreajustarse. MaxPooling también ayuda a que el modelo sea menos sensible a pequeños cambios o movimientos en la imagen, útil para clasificación de objetos.

III. RESULTADOS EXPERIMENTALES

Se presentan los resultados tras 90 épocas de entrenamiento, evaluando precisión y eficiencia computacional.

III-A. Comparación de Precisión (Accuracy)

La precisión en el conjunto de prueba es la métrica principal para evaluar la capacidad de generalización de cada modelo.

Modelo	Framework	Mejor Precisión (Test)	Diferencia
DNN	PyTorch	63.27 %	+6.16 %
DNN	TensorFlow	57.11 %	-
CNN	PyTorch	89.18 %	+5.25 %
CNN	TensorFlow	83.93 %	-

Cuadro I

COMPARACIÓN DE PRECISIÓN ENTRE MODELOS Y FRAMEWORKS.

III-B. Comparación de Eficiencia (Tiempo de Entrenamiento)

Se midió el tiempo total de entrenamiento para evaluar eficiencia computacional, considerando que la velocidad también es un factor importante al diseñar y probar modelos de deep learning.

Modelo	Framework	Tiempo Total (s)	Velocidad Relativa
DNN	PyTorch	3504.87	1x (Base)
DNN	TensorFlow	360.68	~9.7x más rápido
CNN	PyTorch	794.82	1x (Base)
CNN	TensorFlow	204.19	~3.9x más rápido

Cuadro II

COMPARACIÓN DE TIEMPOS DE ENTRENAMIENTO.

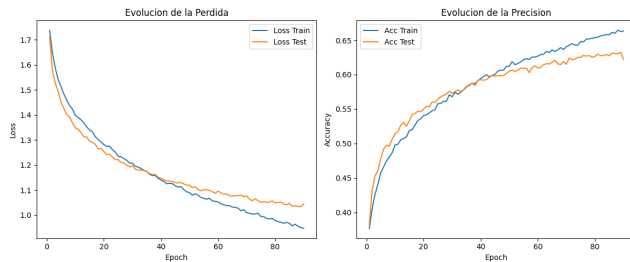


Figura 1. Curvas de aprendizaje para el modelo DNN (Pérdida y Precisión). Se observa convergencia rápida, pero con una brecha significativa entre entrenamiento y validación, indicando cierta sobreajuste.

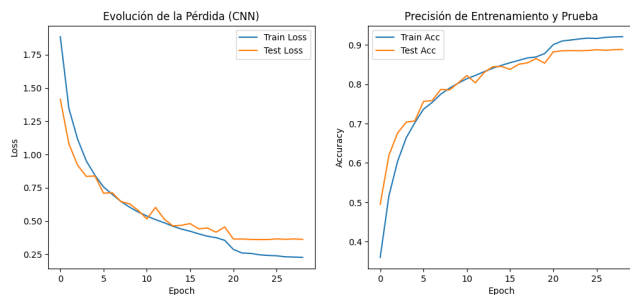


Figura 2. Evolución de la función de pérdida y precisión para el modelo CNN. La generalización es notablemente superior al modelo DNN, mostrando estabilidad durante el entrenamiento.

IV. ANÁLISIS COMPARATIVO DE FRAMEWORKS

El análisis muestra diferencias claras entre PyTorch y TensorFlow:

1. **Precisión (ventaja de PyTorch):** PyTorch alcanzó aproximadamente 5 a 6 % más de precisión en ambos modelos, probablemente debido a pequeñas diferencias en inicialización de pesos, manejo del learning rate y cálculos internos de los frameworks.
2. **Velocidad (ventaja de TensorFlow):** TensorFlow entrenó significativamente más rápido, lo que permite iterar más modelos y ajustar hiperparámetros de manera eficiente.

V. CONCLUSIÓN

La elección entre una arquitectura DNN y CNN para CIFAR-10 es evidente: la **CNN es claramente superior**, alcanzando cerca del 90 % de precisión frente al 60 % de la DNN. Esto se debe a que la CNN puede aprovechar la estructura espacial de las imágenes para aprender características jerárquicas.

En cuanto a los frameworks:

- Si se busca la **máxima precisión**, **PyTorch** es la mejor opción.
- Si la prioridad es la **velocidad y rapidez de entrenamiento**, **TensorFlow** permite realizar más experimentos en el mismo tiempo.

En general, se recomienda usar la arquitectura **CNN con MaxPooling implementada en PyTorch** para el despliegue final, ya que ofrece el mejor rendimiento en la métrica de precisión y un aprendizaje estable.

RECURSOS Y CÓDIGO FUENTE

El código completo de los experimentos está disponible en Google Colab:
<https://colab.research.google.com/drive/1KVVk5P8rnw2tctLY5oG1vrEHwyMdCeC?usp=sharing>



Figura 3. Escanea el QR para acceder al proyecto o resultados completos.